WEB PROGRAMMING I · LECTURE 5

LAYOUT SYSTEMS: FLEXBOX

& CSS VARIABLES

From boxes → to systems

RECAP

UNTIL NOW

We styled elements.

TODAY

We control layout.

THE PROBLEM

YOU HAVE 3 ELEMENTS.

Using only margins, try to:

- Align them to the center
- Space them evenly
- Make it responsive

Margins are hacks.

Layout ≠ spacing

SECTION 1 — FLEXBOX INTRO

FLEXBOX = LAYOUT SYSTEM

The browser helps you arrange elements.

CORE CONCEPT

1 Container

→ controls →

Item | Item | Item

MANY ITEMS

ACTIVATION

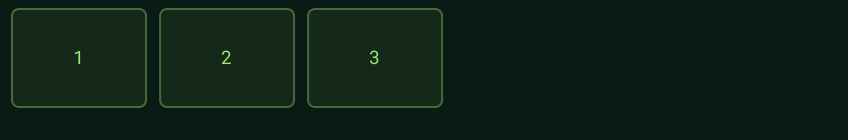
```
.container {
  display: flex;
}
```

This changes everything.

SECTION 2 — AXIS THINKING

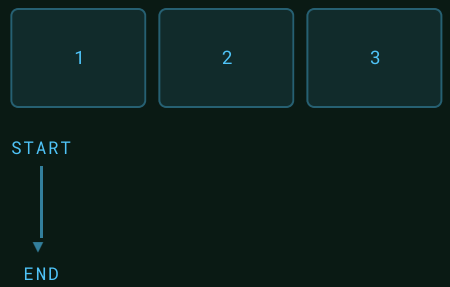
TWO AXES

MAIN AXIS



Items flow along this axis.

CROSS AXIS



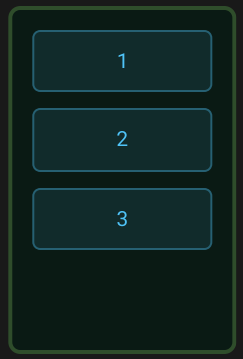
Perpendicular to main axis.

FLEX-DIRECTION

ROW (DEFAULT)

```
flex-direction: row;
```

COLUMN




```
flex-direction: column;
```

Defines the main axis direction.

Direction = flow.

Know your direction first.

Everything else follows.

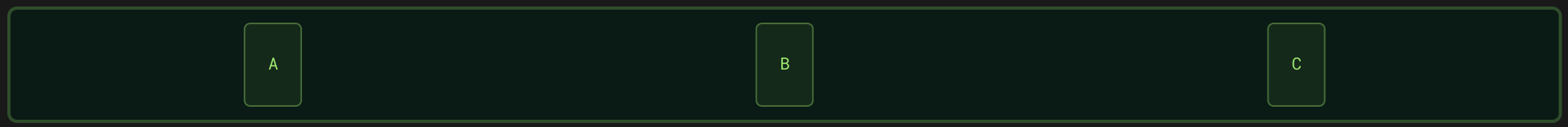
SECTION 3 — ALIGNMENT | JUSTIFY-CONTENT

MAIN AXIS ALIGNMENT

CENTER

SPACE-BETWEEN

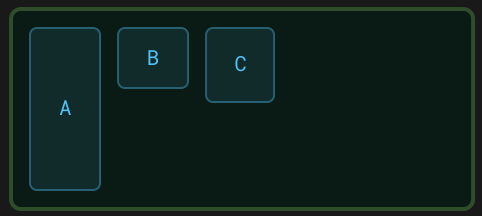
SPACE-AROUND



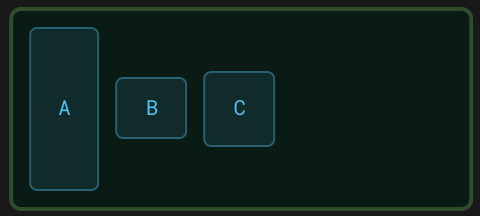
ALIGN-ITEMS

CROSS AXIS ALIGNMENT

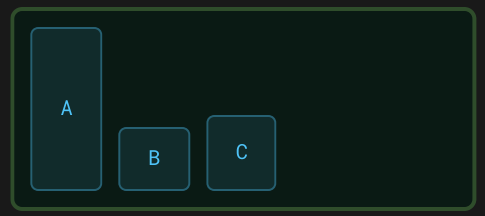
FLEX-START



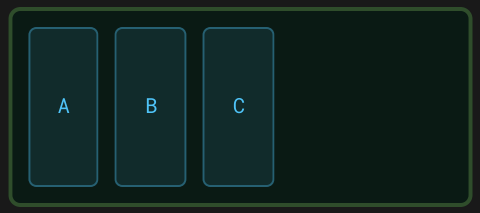
CENTER



FLEX-END



STRETCH



Items have different heights intentionally — watch how each value responds.

2 directions.

You must always know
where your axis is.

SECTION 4 — FLEX ITEMS

ITEMS ARE NOT PASSIVE.

So far: the container gives orders.

Now: items can negotiate.

FLEX-GROW

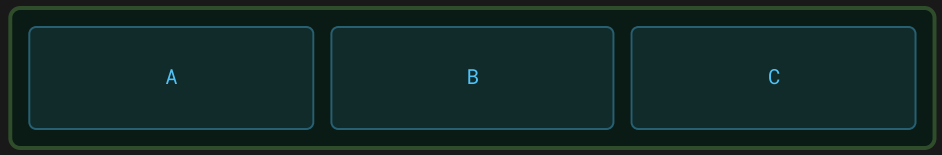
```
.item {
  flex-grow: 1;
}
```

Take all available space.

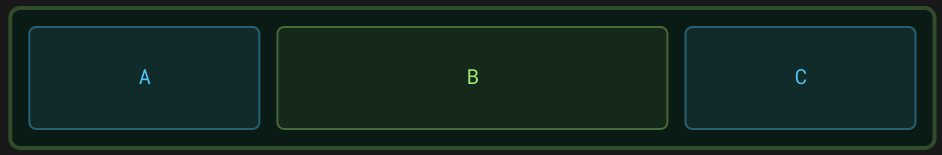
Think of it as a weight.

DEFAULT (FLEX-GROW: 0)

ALL ITEMS: FLEX-GROW 1



B HAS FLEX-GROW: 2



FLEX-SHRINK

```
/* default: shrinks proportionally */
.item {
  flex-shrink: 1;
}

/* refuses to shrink */
.fixed {
  flex-shrink: 0;
}
```

flex-shrink: 1

Items compress proportionally when the container runs out of space.

flex-shrink: 0

Item holds its size no matter what. Useful for logos, icons, and fixed sidebars.

FLEX-BASIS

```
.item {
  flex-basis: 200px;
}

.item {
  flex-basis: 30%;
}

.item {
  flex-basis: auto; /* default */
}
```

flex-basis sets the starting size of an item —

before flex-grow or flex-shrink are applied.

Think of it as the item's "ideal size"
before the negotiation begins.

SHORTHAND

```
/* flex: grow  shrink  basis */
.item {
  flex: 1 1 auto;
}

/* The one shorthand you'll use the most */
.item {
  flex: 1;
  /* same as: flex: 1 1 0 */
}
```

flex: 1 = grow equally · shrink equally · start from zero

Items share all available space evenly. The most common pattern in real UIs.

SECTION 5 — REAL USE

WHERE YOU SEE FLEXBOX EVERY DAY

Navbar

Logo left · links right

`justify-content: space-between`

Cards in a row

Equal width, equal height

`flex: 1 · align-items: stretch`

Center anything

Both axes at once

`justify-content: center · align-items: center`

Sidebar layout

Fixed sidebar · flexible content

`flex: 0 0 250px · flex: 1`

You are not
placing elements.

You are designing
systems.

SECTION 6 — CSS VARIABLES | THE PROBLEM

HARDCODED VALUES EVERYWHERE

```
/* Your brand color is #9FE870 */
.button        { background:   #9FE870; }
.link          { color:        #9FE870; }
.border-accent { border-color: #9FE870; }
.heading       { color:        #9FE870; }

/* Client calls. Wants to change the color.
   Find & replace across 200 files.
   Miss one. Bug ships to production. */
```

THE SOLUTION

Variables in CSS.

Also called Custom Properties.

DEFINE

```
:root {
  --primary:    #9FE870;
  --secondary:  #4FC3F7;
  --bg:         #0B0B0B;
  --text:       #FFFFFF;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --radius:     6px;
}
```

:root is the top-level element — the entire document.

Variables defined here are available in every selector, everywhere.

Syntax rule: name must begin with `--`

USE

```
.button {
  background: var(--primary);
  color:      var(--bg);
  padding:    var(--spacing-sm) var(--spacing-md);
  border-radius: var(--radius);
}

.link {
  color: var(--secondary);
}

.card {
  background: var(--bg);
  color:      var(--text);
  padding:    var(--spacing-md);
  border-radius: var(--radius);
}
```

WHY IT MATTERS

```
:root {
  --primary: #9FE870;
}
```

CHANGE ONCE

→

- button background
- link color
- border accent
- heading color

UPDATE EVERYWHERE

Variables =
Design System.

You are not writing numbers.

You are naming things.

SECTION 7 — WRAP UP

WHAT WE COVERED TODAY

FLEXBOX

- Container vs. Items
- `display: flex` activates it
- Main axis & cross axis
- `justify-content` — main axis
- `align-items` — cross axis
- `flex-grow`, `flex-shrink`, `flex-basis`
- `flex: 1` — the essential shorthand

CSS VARIABLES

- Define in `:root` with `--name`
- Use with `var(--name)`
- Change once, update everywhere
- Names carry design intent
- Foundation of any design system

From static pages
→ flexible systems.

PRACTICE

NOW BUILD REAL LAYOUTS.

ASSIGNMENT

Using Flexbox, build these three patterns from scratch:

- A **navbar** — logo on the left, links on the right
- A row of **3 equal cards** — same width, same height
- A **centered hero** — content perfectly centered both ways